Module 7 – Advanced PHP Excercises

-BARAD VIPUL

1. Encapsulation

Code:

```
<?php
class Student {
   private $name;
   public function setName($name){ $this-
>name=$name; }
   public function getName(){ return $this->name;
}
}
$obj=new Student();
$obj->setName("Vipul");
echo $obj->getName();
?>
```

Explanation:

Private property accessed using getter/setter methods.

2. Car Class

Code:

```
<?php
class Car {
   public $make,$model,$year;
   function __construct($m,$mo,$y){
      $this->make=$m; $this->model=$mo; $this-
>year=$y;
   }
   function display(){
      echo "$this->make $this->model $this->year";
   }
}
?>
```

Explanation:

Defines car properties and method to display details.

3. Multiple Objects

Code:

```
<?php
$c1=new Car("Toyota","Fortuner",2022);
$c2=new Car("Honda","City",2021);
$c1->display();
echo "<br>";
$c2->display();
?>
```

Explanation:

Creates multiple objects and calls methods.

4. Inheritance

Code:

```
<?php
class Vehicle {
   function start(){ echo "Vehicle started<br>"; }
}
class Car extends Vehicle {
   function drive(){ echo "Car driving"; }
}
$obj=new Car();
$obj->start();
$obj->drive();
?>
```

Explanation:

Car inherits Vehicle methods.

5. Overloading

Code:

```
<?php
class Test {
   function __call($name,$args){
      echo "Called $name with ".count($args)."
args";
   }
}
$obj=new Test();
$obj->sum(1,2);
?>
```

Explanation:

Simulates overloading using __call().

6. Interface

Code:

```
<?php
interface VehicleInterface{
   public function start();
   public function stop();
}
class Bike implements VehicleInterface{
   function start(){ echo "Start"; }
   function stop(){ echo "Stop"; }
}
?>
```

Explanation:

Interface ensures method implementation.

7. Constructor

Code:

```
<?php
class User{
   public $name;
   function __construct($name){ $this-
>name=$name; }
}
$obj=new User("Vipul");
?>
```

Explanation:

Constructor initializes object properties.

8. Destructor

Code:

```
<?php
class Demo{
   function __destruct(){ echo "Destroyed"; }
}
$obj=new Demo();
?>
```

Explanation:

Destructor runs at script end.

9. Magic Methods

Code:

```
<?php
class Magic{
   private $data=[];
   function __set($n,$v){ $this->data[$n]=$v; }
   function __get($n){ return $this->data[$n]; }
}
$obj=new Magic();
$obj->name="Vipul";
echo $obj->name;
?>
```

Explanation:

Handles dynamic properties.

10. Static

Code:

```
<?php
class Test{
   static $num=10;
   static function show(){ echo self::$num; }
}
Test::show();
?>
```

Explanation:

Access static using :: operator.

11. Traits

Code:

```
<?php
trait A{ function a(){ echo "A"; } }
trait B{ function b(){ echo "B"; } }
class C{ use A,B; }
$obj=new C();
$obj->a(); $obj->b();
?>
```

Explanation:

Traits reuse code.

12. Visibility

Code:

```
<?php
class Demo{
   public $a=1;
   private $b=2;
   protected $c=3;
}
?>
```

Explanation:

Shows access levels.

13. Type Hint

Code:

```
<?php
function add(int $a,int $b){ return $a+$b; }
echo add(2,3);
?>
```

Explanation:

Ensures correct data types.

14. Final

Code:

```
<?php
final class A{}
class B extends A{} // error
?>
```

Explanation:

Final class cannot be extended.

15. Email Validate

Code:

```
<?php
$email="test@gmail.com";
$email=filter_var($email,FILTER_SANITIZE_EMAI
L);
if(filter_var($email,FILTER_VALIDATE_EMAIL)){
 echo "Valid";
}
?>
```

Explanation:

Sanitizes and validates email.

16. File Read

Code:

```
<?php
$f=fopen("test.txt","r");
echo fread($f,filesize("test.txt"));
fclose($f);
?>
```

Explanation:

Reads file content.

17. Mail

Code:

```
<?php
mail("test@gmail.com","Test","Hello");
?>
```

Explanation:

Sends email.

18. MVC

Code:

```
<?php
// Controller
include "model.php";
$data=getData();
include "view.php";
?>
```

Explanation:

Separates logic, data, UI.

19. Mini Project

Code:

```
<?php
// Example structure only
class Book{}
class User{}
?>
```

Explanation:

Use OOP concepts in project.

20. DB Connect

Code:

```
<?php
$conn=mysqli_connect("localhost","root","","test")
;
if(!$conn){ die("Error"); }
?>
```

Explanation:

Connects database.

21. SQL Injection

Code:

```
<?php
// safe
$stmt=$conn->prepare("SELECT * FROM users
WHERE id=?");
$stmt->bind_param("i",$id);
?>
```

Explanation:

Prevents injection.

22. Try Catch

Code:

```
<?php
try{
 $pdo=new
PDO("mysql:host=localhost;dbname=test","root","
");
}catch(Exception $e){
 echo $e->getMessage();
}
?>
```

Explanation:

Handles exceptions.

23. Validation

Code:

```
<?php
if(!preg_match("/^[a-zA-Z]+$/",$name)){
 echo "Invalid";
}
?>
```

Explanation:

Validates input.

24. Email Send

Code:

```
<?php
mail($email,"Welcome","Success");
?>
```

Explanation:

Sends confirmation email.

25. Session Cookie

Code:

```
<?php
session_start();
$_SESSION['user']="Vipul";
setcookie("user","Vipul",time()+3600);
?>
```

Explanation:

Stores user data.

26. Upload

Code:

```
<?php
move_uploaded_file($_FILES['f']['tmp_name'],"uploads/".$_FILES['f']['name']);
?>
```

Explanation:

Uploads file.

27. CRUD

Code:

```
<?php
mysqli_query($conn,"INSERT INTO users(name)
VALUES('Vipul')");
?>
```

Explanation:

Basic CRUD operation.